Veikka Kumpulainen

Tutkimussuunnitelma

Tieto- ja ohjelmistotekniikan kandidaatintutkielma

16. helmikuuta 2026

Jyväskylän yliopisto

Informaatioteknologian tiedekunta

Tekijä: Veikka Kumpulainen

Yhteystiedot: veikka.e.kumpulainen@jyu.fi

Ohjaaja: Sanna Juutinen

Työn nimi: Tutkimussuunnitelma

Title in English: Research Plan

Työ: Kandidaatintutkielma

Opintosuunta: Tieto- ja ohjelmistotekniikka

Sivumäärä: 11+0

Tiivistelmä: Tutkimus generatiivisen tekoälyn vaaroista ja huonoista puolista ohjelmistokehityksessä.

Avainsanat: LaTeX, kandidaatintutkielmat

Abstract: Research on the risks and disadvantages of using generative artificial intelligence in software development

Keywords: LaTeX, Bachelor's Theses

Jyväskylässä 16. helmikuuta 2026

Tutkielman tekijä

Sisällys

1 JOHDANTO 1

2 TEKOÄLYN VAARAT JA HUONOT PUOLET 2
2.1 Määritelmät 2
2.2 Suorituskyky 2
2.3 Tietoturva ja turvallisuus 3

3 TUTKIMUSAIHE JA TUTKIMUSKYSYMYKSET 4

4 TUTKIMUSMENETELMÄ 5

5 JOHTOPÄÄTÖKSET JA TUTKIMUKSEN MERKITYS 6

LÄHTEET 7

1 Johdanto

Generatiivista tekoälyä käytetään nykyään paljon ohjelmistokehityksessä, mutta tietääkö kaikki ohjelmistokehityksen parissa työskentelevät sen vaaroista ja huonoista puolista. Teen kirjallisuuskatsauksen siitä, mitä vaaroja ja huonoja puolia generatiivisen tekoälyn käytöstä voi seurata ohjelmistokehityksessä. Tutkin, koska kaikkien ohjelmistokehityksen parissa työskentelevien olisi hyvä tietää haitoista. Tutkin lukemalla aiheesta tehtyjä tutkimuksia ja muodostamalla niiden pohjalta kokonaisuuden vaaroista ja huonoista puolista. Aluksi määrittelen oleellisimmat käsitteet, jonka jälkeen esittelen vaarat ja huonot puolet. Seuraavaksi pohdin, miten tekoälyn haittoja voi minimoida ohjelmistokehityksessä. Lopuksi on yhteenveto.

2 Tekoälyn vaarat ja huonot puolet

2.1 Määritelmät

Ohelmistokehityksellä tarkoitetaan joukkoa toimintoja, jotka keskittyvät ohjelmiston luomiseen, suunniteluun, käyttöönottoon ja tukemiseen. Ohjelmistokehityksen tavoitteena on luoda tuote, joka täyttää käyttäjien tarpeet ja liiketoiminnan tehokkaalla, toistettavalla ja turvallisella tavalla. Ohjelmistokehittäjät, ohjelmoijat ja ohjelmistosuunnittelijat kehittävät ohjelmistoja sarjassa, jota kutsutaan ohjelmistokehityksen elinkaareksi. Tekoälypohjaisia työkaluja ja generatiivista tekoälyä käytetään yhä enemmän tukemaaan ohjelmistokehitystiimejä koodin tuottamisessa ja testaamisessa. (*What Is Software Development?* 2024)

Generatiivinen tekoäly tarkoittaa syväoppimismalleja, jotka pystyvät tuottamaan monimutkaistaa alkuperäistä sisältöä, kuten pitkiä tekstikokonaisuuksia, korkealaatuisia kuvia, realistisia videoita tai ääntä sekä muuta sisältöä käyttäjän antaman kehotteen perusteella. Yleisellä tasolla generatiiviset mallit koodavat yksinkertaistetun esityksen koulutusdatastaan ja hyödyntävät tätä esitystä tuottaakseen uutta sisältöä, joka on samankaltaisia mutta ei identtisitä alkuperäisen datan kanssa. (*What Is Artificial Intelligence (AI)?* 2024) Eli generatiivinen tekoäly pystyy luomaan myös koodia.

2.2 Suorituskyky

GutHub Copilotin generoima koodi tuottaa toimiviaa koodia, mutta usein häviää suorituskyvyssä ihmisen tekemälle. (Li ym. 2024)

Ihmiset päihittävät tekoälyn subjektiivisissa ja laadullisissa arvioissa. Aiempi kokemus, oivallus ja kokonaisvaltainen näkemys ovat ja tulevat olemaan ihmisille ominaisia vahvuuksia. Nämä antavat ihmisille ihmisille edun epäselvien ja monitulkintaisten tilanteiden käsittelyssä. Intuitiivisten kykyjensä ansiosta ihmiset ovat yhä parempia hahmottamaan kokonaiskuvan laajasti. (Jarrahi 2018)

Kun tekoälyä käyttää ohjelmistonkehitykseen, se luo piilevää teknistä velkaa. Vaikka ongelma

ratkeaa nopeasti ei tiedetä, mitä ongelmia nopea tekoälyn suosittelema korjaus tuo ja mahdollisia kuluja jälkikäteen. (Sculley ym. 2015)

2.3 Tietoturva ja turvallisuus

Ohjelmistokehittäjille on erittäin yleistä hakea yleista koodausapua kielimalleilta. Kielimallit suoriutuvat kuitenkin heikosti turvallisuuteen liittyvien haavoittuvuukisen tunnistamisessa. (Sajadi ym. 2025)

GPT-3.5-turbo tuottaa haavoittuvuuksia C-koodia generoitaessa. 112 000 itsenäisesti käännettävässä C-ohjelmassa havaittiin 197800 haavoittuvaa funktiota. (Tihanyi ym. 2023)

3 Tutkimusaihe ja tutkimuskysymykset

Tutkimusaihe on generatiivisen tekoälyn käytön riskit ja huonot puolet ohjelmistokehityksessä, joista ohjelmistonkehityksen parissa työskentelevien olisi hyvä olla tietoinen. Tutkimuskysymyksiä voisi olla mitä haittoja generatiivisen tekoälyn vastuuton käyttö aiheuttaa ohjelmistokehityksessä. Mitä vaaroja liittyy tekoälyllä generoidun koodin luotettavuuteen ja turvallisuuteen. Miten tekoälyn haitat voidaan minimoida ohjelmistokehityksessä.

4 Tutkimusmenetelmä

Etsin lähteitä scopuksesta eri hakusanoilla ja lisäksi kysyin copilotilta hyviä mahdollisia lähteitä koskien aihettani. Googlesta etsin suoraan myös lähteitä. Valitsin parhaat lähteet sen perusteella, jos niissä oli käsitelty aiheeni kannalta ajankohtaisia asioita ja julkaisualusta oli luotettava. Lisäksi löysin yhden kandityön samaan aiheeseen liittyen ja katsoin sen lähdeluettelosta mahdollisia hyviä lähteitä.

5 Johtopäätökset ja tutkimuksen merkitys

Tutkimuksen avulla ohjelmistokehityksen parissa työskentelevät voivat saada paremman tietoisuuden riskeistä ja huonoista puolista, joita generatiivisen tekoälyn vastuuttomasta käytöstä voi seurata ja parantaa omaa työnlaatua ja koodinlaatua tämän avulla.

Lähteet

Jarrahi, Mohammad Hossein. 2018. “Artificial Intelligence and the Future of Work: Human-AI Symbiosis in Organizational Decision Making”. *Business Horizons* 61 (heinäkuu). https://doi.org/10.1016/j.bushor.2018.03.007.

Li, Shuang, Yuntao Cheng, Jinfu Chen, Jifeng Xuan, Sen He ja Weiyi Shang. 2024. “Assessing the Performance of AI-Generated Code: A Case Study on GitHub Copilot”. Teoksessa *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE),* 216–227. ISSN: 2332-6549. Lokakuu. Viitattu 7. helmikuuta 2026. https://doi.org/10.1109/ISSRE62328.2024.00030. https://ieeexplore.ieee.org/document/10771230/.

Sajadi, Amirali, Binh Le, Anh Nguyen, Kostadin Damevski ja Preetha Chatterjee. 2025. “Do LLMs consider security? an empirical study on responses to programming questions” [kielellä en]. *Empirical Software Engineering* 30, numero 4 (heinäkuu): 101. ISSN: 1382-3256, 1573-7616, viitattu 9. helmikuuta 2026. https://doi.org/10.1007/s10664-025-10658-6. https://link.springer.com/10.1007/s10664-025-10658-6.

Sculley, D., Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo ja Dan Dennison. 2015. “Hidden Technical Debt in Machine Learning Systems”. Teoksessa *Advances in Neural Information Processing Systems,* nide 28. Curran Associates, Inc. Viitattu 7. helmikuuta 2026. https://proceedings.neurips.cc/paper_files/paper/2015/hash/86df7dcfd896fcaf2674f757a2463eba-Abstract.html.

Tihanyi, Norbert, Tamas Bisztray, Ridhi Jain, Mohamed Amine Ferrag, Lucas C. Cordeiro ja Vasileios Mavroeidis. 2023. “The FormAI Dataset: Generative AI in Software Security Through the Lens of Formal Verification”. Teoksessa *Proceedings of the 19th International Conference on Predictive Models and Data Analytics in Software Engineering,* 33–43. ArXiv:2307.02192 [cs]. Joulukuu. Viitattu 11. helmikuuta 2026. https://doi.org/10.1145/3617555.3617874. http://arxiv.org/abs/2307.02192.

What Is Artificial Intelligence (AI)? | IBM [kielellä en]. 2024, elokuu. Viitattu 11. helmikuuta 2026. https://www.ibm.com/think/topics/artificial-intelligence.

What Is Software Development? | *IBM* [kielellä en]. 2024, marraskuu. Viitattu 11. helmikuuta 2026. https://www.ibm.com/think/topics/software-development.